

Takokit — Local Voice AI Runtime

Core Idea

Takokit is a local-first voice AI runtime, similar in spirit to Ollama, but built for speech and voice models.

It lets users:

- run open-source TTS models locally
- clone voices from short samples
- train/fine-tune custom voices
- transcribe audio using Whisper
- convert voices
- expose a local API for other apps
- manage everything through a desktop UI and CLI

One-Line Pitch

Takokit is a local voice AI runtime for running, cloning, training, and serving open-source voice models.

Tagline

Run open voices locally.

Name Logic

Takokit comes from the TacoKit/Tacotron direction:

Tacotron → Taco/Tako → Takokit

It keeps the technical connection to TTS architecture, but sounds simpler, more memorable, and easier to brand.

Product Positioning

Ollama → local LLMs
Takokit → local voice models

Expanded version:

Ollama + Whisper + local ElevenLabs-style voice cloning

Main Features

1. Local Text-to-Speech

Generate speech locally from text using open-source models.

Example:

```
takokit speak "Hello, this is running locally."
```

2. Voice Cloning

Upload or record a short voice sample and generate speech in that voice.

Example:

```
takokit clone ./sample.wav --name amaan  
takokit speak "Welcome back" --voice amaan
```

3. Voice Training

Users can provide more audio data for better custom voices.

Pipeline:

```
audio samples → Whisper transcription → cleaned dataset → fine-tuned voice
```

4. Speech-to-Text

Whisper is included as a support engine.

```
speech/audio → text
```

Used for:

- transcription
- dataset creation
- dubbing

- live assistant input
- speech-to-speech workflows

5. Local API Server

Developers can run:

```
takokit serve
```

Then call:

```
POST http://localhost:5050/v1/audio/speech
```

The API should be OpenAI-compatible where possible.

6. Desktop UI

The app should include:

- model store
- installed models page
- voice library
- speak/test screen
- voice cloning wizard
- voice training wizard
- API server dashboard
- logs and hardware status

Suggested Architecture

```
Desktop UI / CLI
  ↓
Takokit Local Server
  ↓
Model Manager + Voice Registry
  ↓
Engine Adapters
  ↓
Kokoro / Chatterbox / GPT-SoVITS / Qwen3-TTS / Whisper / RVC
  ↓
Audio Output / Transcript / API Response
```

Core Modules

Takokit Desktop

Built with:

```
Tauri + React + TypeScript
```

Takokit Server

Built with:

```
Python + FastAPI
```

Handles:

- model loading
- inference
- voice cloning
- training jobs
- transcription
- API requests

Takokit CLI

Possible commands:

```
takokit serve
takokit pull kokoro
takokit pull chatterbox
takokit speak "Hello"
takokit clone ./voice.wav --name myvoice
takokit train ./samples --name myvoice-v2
takokit transcribe ./meeting.mp3
```

Model Registry

Stores model metadata:

```
{
  "id": "kokoro",
  "type": "tts",
  "license": "apache-2.0",
```

```
"supports_cloning": false,  
"supports_training": false,  
"hardware": "cpu-friendly"  
}
```

Engine Adapter Layer

Every model is wrapped behind one interface:

```
speak(text, voice, settings)  
clone(audioSample, name)  
train(dataset, config)  
transcribe(audio)  
convertVoice(audio, targetVoice)
```

Recommended MVP Model Stack

MVP 1

```
Kokoro      → fast local TTS  
Whisper     → transcription  
Chatterbox  → voice cloning  
GPT-SoVITS  → few-shot voice training
```

MVP 2

```
Qwen3-TTS   → voice design + streaming  
RVC         → voice conversion  
Piper       → lightweight offline voices
```

Later

```
CosyVoice  
Fish Speech  
F5-TTS  
IndexTTS  
VibeVoice  
Dia  
OpenVoice  
Bark  
MeloTTS
```

Tech Stack

Desktop:	Tauri + React + TypeScript
Backend:	Python + FastAPI
ML Runtime:	PyTorch first, ONNX Runtime later
Audio Tools:	FFmpeg
ASR:	Whisper / whisper.cpp
Database:	SQLite
CLI:	Rust or Go

Local Storage

```
~/.takokit/  
  models/  
  voices/  
  datasets/  
  cache/  
  logs/  
  config.json
```

MVP Build Plan

Phase 1 — Core Runtime

- Create CLI
- Create local FastAPI server
- Add `/v1/audio/speech`
- Add Kokoro support
- Generate `.wav` output

Phase 2 — Desktop UI

- Model list
- Text-to-speech screen
- Voice selector
- Server status
- Audio preview/export

Phase 3 — Whisper Integration

- Add transcription
- Add voice sample transcription
- Add dataset preparation flow

Phase 4 — Voice Cloning

- Add Chatterbox or GPT-SoVITS
- Upload sample
- Generate cloned speech
- Save voice profile

Phase 5 — Training Workflow

- Upload multiple samples
- Clean/transcribe samples
- Start training/fine-tuning job
- Show progress
- Save trained voice

Phase 6 — Developer Experience

- OpenAI-compatible API
- SDK examples
- Local docs page
- Model registry
- Better logs and error handling

Timeline Estimate

For a small beginner team:

CLI + server + Kokoro:	2–3 weeks
Desktop UI:	2–3 weeks
Whisper + dataset tools:	1–2 weeks
Voice cloning MVP:	3–5 weeks
Training workflow:	3–5 weeks
Stable beta:	8–12 weeks total
Polished public version:	4–6 months

Main Risks

1. Model setup complexity

TTS models often have fragile Python, CUDA, PyTorch, and FFmpeg dependencies.

2. Licensing

Some models are open-source but not commercial-safe. Takokit needs clear license labels per model.

3. Voice cloning misuse

The app needs consent warnings, safe defaults, and possibly watermarking or metadata logs.

4. Hardware limitations

Some models will run well on CPU. Others will need GPU.

5. Quality inconsistency

Short-sample cloning can vary depending on mic quality, noise, accent, and sample length.

Final Positioning

Takokit is a local-first voice AI runtime that lets users run, clone, train, transcribe, convert, and serve open-source voice models from one desktop app, CLI, and local API.